

LoFSI

Logique et Fondements des Systèmes Intelligents

I. Préface

A. Disclaimer

- Année particulière ; les cours sont en train d'être conçus. Grosse charge liée à IMSV + département.

⚠ Le résultat risque d'être "bancal".

😊 Pas de panique ! Je perds de mon mieux pour que ça se passe bien et que ça soit intéressant !

↔ Les choses vont se clarifier au fil des semaines.

⚠️ Votre avis est important,
d'autant plus dans cette année
charnière !

B. Philosophie du cours

Preuve



Modèle



Calcul



Mécanisat°



Vérificat°

Nous sommes ici

IntLog. Qu'est-ce qu'un raisonnement mathématique?
Quelle en est la puissance?
Quelles en sont les limites?

LoFSI. Quelles formes de raisonnement peut-on rendre effectives, à quel coût et avec quelles garanties?

FMSA. Comment contrôler un agent autonome et prouver qu'il est sûr?

LoFSI - Objectif

Explorer les aspects effectifs de la logique : automates, décidabilité, calculabilité, complexité. Ouvrir sur les systèmes intelligents et l'utilisat° pratique de la logique.

Interlude

J'aimerais faire le bilan de ce que vous avez vu en logique, par rapport à ce que j'aurais aimé y voir.

Deux raisons :

- 1) Mieux cerner ce que Christian faisait.
- 2) Adapter LFSI à ce que vous avez réellement vu.

Procédure :

- 1) Je vous montre le ToC.
- 2) Pour chaque point, vous donner votre niveau de confiance via Woodley, (anonyme)
- 3) Discussion ouverte tout du long.

TOC (à haut niveau)

- 1) Logique propositionnelle : syntaxe et sémantique.
- 2) Systèmes de déduct° : déduct° naturelle, systèmes de Hilbert, séquenti...
- 3) Correct°, complétude et décidabilité de la logique propositionnelle.
- 4) Logique du 1^{er} ordre : correct°, consistance
- 5) Complétude de Gödel
- 6) Compacité
- 7) Löwenheim - Skolem, modèles non-standard

8) ZF, axiome du choix, Zorn,
Zornelo, etc.

9) Incomplétude de Gödel.

TOC de LFS1

Q1: Calculabilité et mécanisme du
raisonnement logique

1) Logique, langage et calcul.

2) Langages et automates.

3) Machines de Turing et fonctions
calculables

4) Décidabilité

5) Complexité

6) Logiques Décidables

Q2: raisonner avec des outils

7) SAT et SMT

8) λ -calcul et types

9) Lean en pratique

10) Introduction aux LLMs.

!! La ToC va certainement évoluer.

C. Support

• Principalement, cours au tableau.

• Plusieurs livres de référence disponibles.

↳ Je vais essayer d'avoir un syllabus à terme, mais probablement compliqué cette année.

😊 OK de donner mes notes mois
écriture d'élève.
+ Non contractuelles !

⇒ C'est ce qui est fait en classe
qui compte.

D. Evaluations

Q1 | Examen oral avec une partie
à cours ouvert (phase de
préparation). S'illustrerai les
attentes au fil de l'année.

Q2 | "Atelier aux astéroïdes".

→ Long à écrire, j'expliquerai
en direct.

Note finale :

- Si $Q1$ ou $Q2 < 7$, alors $\min \{Q1, Q2\}$.
- Sinon $50\% \cdot Q1 + 50\% \cdot Q2$.

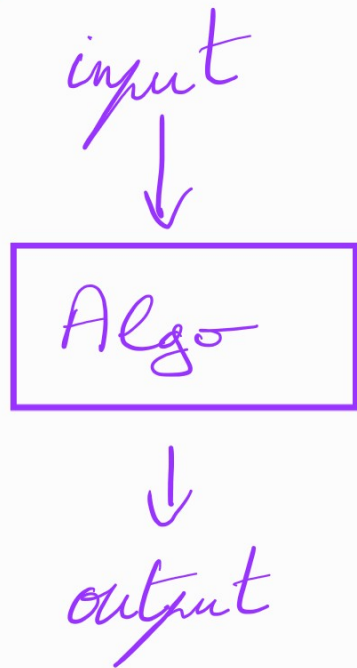
⚠️ Il s'écrit formellement les modalités du cours : liiez le fichier sur Moodle !

E. Introduction

On veut comprendre quels types de raisonnement peuvent être "mécanisés".
Autrement dit, être rendus effectifs, via des algorithmes.

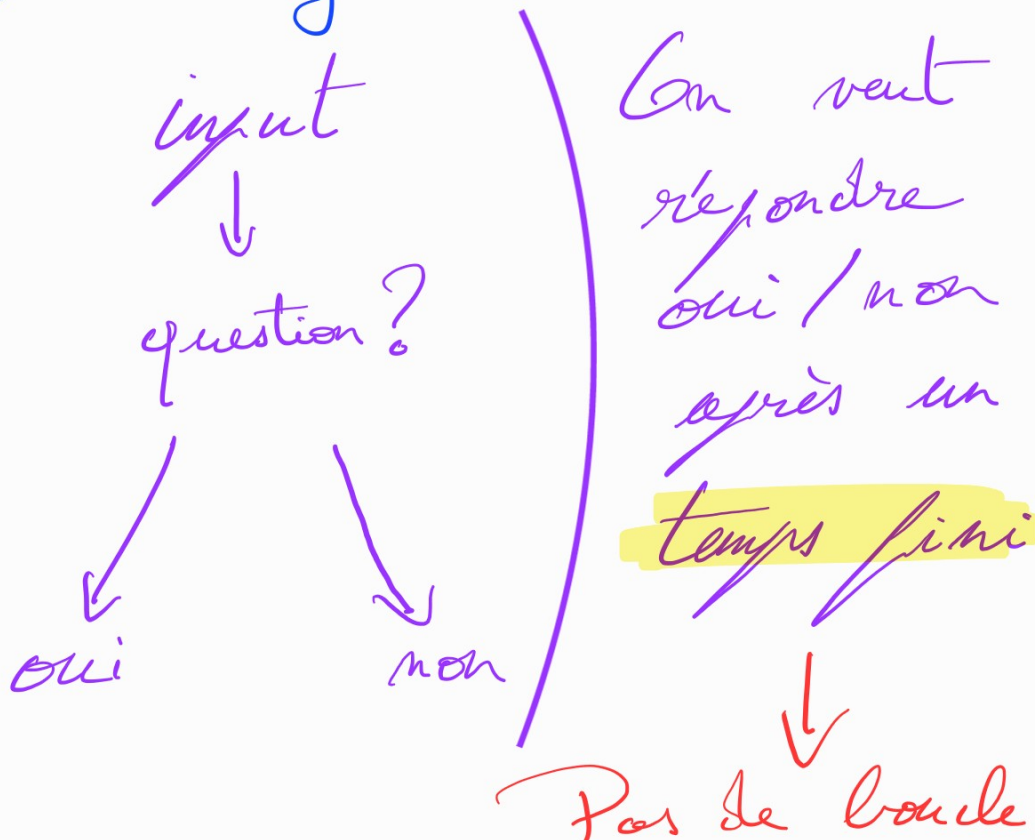
Il faut donc formaliser la notion d'algorithme.

Algorithme



Procédure effective
décrite sans
ambiguïté

Un algorithme répond à un problème.
Classiquement, on considère un
problème de décision. On parle
alors d'algorithme décideur.



Plus tard, on distinguera

- **décider** : on termine (s'arrête) tjs, et on répond correctement.
- **reconnaître** : on termine et on accepte si OUI ; on rejette ou on ne termine pas (boucle ∞) si NON.

Input

L'input peut être n'importe quoi :
graphe, formule, nombres, programme, etc.
Pour formaliser l'idée d'algorithme,
nous allons **encoder** l'input comme
un **mot fini** sur un **alphabet** fixé.

Au sens mathématique, c'est
simplement une suite finie de
symboles (**lettres**) appartenant à
l'**alphabet** d'entrée.

Σ alphabet

$$\begin{array}{ccc} x & \mapsto & \text{enc}(x) \in \Sigma^* \\ \text{Input} & & \text{Encodage de } x \end{array}$$

Etoile
de Kleene

$$\Sigma^* = \{ \text{mots finis} \\ \text{construits sur } \Sigma \}$$



Nous y reviendrons

Point crucial:
input finiment
représentable

Ce n'est pas limitant : pensez en
binaire par exemple.

Spoiler

La complexité d'un algorithme se
mesurera en fonction de la taille de
l'encodage.

↳ Ce dernier est donc crucial.

Langages

L'input étant un mot, on peut voir le problème de décision comme un problème de reconnaissance de langage.

Etant donné un mot $w \in \Sigma^*$, décider le problème P revient à décider si $w \in L_P$ où

$$L_P = \{ \text{enc}(x) \mid x \text{ est une instance positive de } P \}$$

Cette vision est utile car on peut manipuler formellement des langages.

On suppose :

- un alphabet Σ fini,
- un encodage non ambigü.

Rmq

Encodage non valide $\Rightarrow$ Non.

Exemple

• Problème : étant donné un mot binaire, contient-il le facteur "101"?

↳ Alphabet $\Sigma = \{0, 1\}$

↳ $L_{101} = \{w \in \Sigma^* \mid w \text{ contient le facteur "101"}\}$

• Quelques instances :

001011 ✓

11100 ✗

ϵ ✗

101 ✓

|

ϵ représente le mot vide : il ne contient aucune lettre et est de longueur zéro.

Quest^o → A discuter au tableau.

Que faut-il mémoriser pendant la lecture d'un mot pour pouvoir décider ?

Exemple beaucoup + complexe

Problème: étant donné un énoncé φ et une preuve candidate π , tous deux encodés dans un système formel fixe S , déterminer si π est une preuve correcte de φ dans S .

Langage:

$$V_S = \{ \langle \varphi, \pi \rangle \mid \pi \text{ est une preuve valide de } \varphi \text{ dans le système } S \}$$

↳ Problème bcp + complexe mais aussi couvert dans notre framework.

⇒ Vérifier si une preuve est correcte est en général décidable.

! Pour un système formel S effectivement présenté, où la validité d'une étape de dérivat° est décidable.

Problème

Un autre problème, plus complexe encore, consiste à décider, étant donné seulement un énoncé φ , s'il existe une preuve de φ dans S .

Langage

$$T'_S = \{ \varphi \mid \exists \text{ une preuve de } \varphi \text{ dans } S \}$$

↳ En général, ce sera reconnaissable (on peut énumérer les preuves) mais pas décidable (pour S suffisamment puissant / expressif).

⇒ Vérifier qu'une "sdut^o" est correcte est souvent + facile que déterminer si une sdut^o existe.

Pour tout système cohérent, effectivement axiomatisé, et suffisamment expressif pour formaliser une portion élémentaire de l'arithmétique, déterminer si un énoncé possède une preuve est indécidable.

C'est étroitement lié aux limites révélées par les théorèmes d'incomplétude de Gödel (cf. IntLog).

↳ Nous y reviendrons.

En pratique, il conviendrait de distinguer :

- 1) la **vérificat°** d'une preuve donnée,
- 2) l'**aide à la construct°** de preuve,
- 3) la "garantie" de pouvoir **décider**, pour tout énoncé φ , s'il existe une preuve de φ dans S ; ce qui est impossible en toute généralité.

Vision logique

Vérité vs.

démontrabilité

Not° sémantique

Not° syntaxique

Les deux peuvent être reliées via **correct°** et **complétude** dans certains contextes.

↳ Un outil comme **Lean**, qui sera étudié au Q2, supporte (1) et (2).

Rmq

Lean travaille par rapport à des assertions déclarées : il ne fournit pas une affirmation absolue et indépendante du cadre formel choisi.

Il faudra aussi distinguer impossibilité théorique (indécidabilité), qui repose sur une quantification universelle des inputs, et ce qu'on peut faire "en pratique" (p.ex., résoudre les cas de cas restreints).

De $\hat{\Omega}_m$, un problème peut être théoriquement décidable, mais en pratique avoir un coût computationnel prohibitif.

↳ On établira une mesure et une hiérarchie de complexité.

↳ Avec cette vision, nous allons nous lancer dans le premier chapitre et considérer un modèle algorithmique simple permettant de manipuler des langages : les automates finis.

ENC - mêmes
simples

FIN